Jira Enhancer: Supplementary Methods and Related Work

Anonymous authors

1 Related-work comparison

Table S1: Comparison of the main paradigms discussed in the background and related work: representative strengths, enterprise-planning limitations, and the position of Jira Enhancer (B3) relative to B0/B1/B2.

Paradigm	Representative prior work	Typical strength	Common constraint in enterprise planning	Position of B0/B1/B2/B3
Template/rule decomposition	RE standards, agile requirements, and user-story quality work [1–3]	High consistency and stable formatting	Little ability to adapt to ambiguous epics; poor semantic understanding of unfamiliar situations	B0 and B1 are closest relatives of this family; B3 retains its structure but adds adaptive semantic reasoning
Generic LLM drafting	Prompt-driven generation, code assistants, and reasoning methods [4–7]	natural-language drafts and broad transfer between tasks	Fluency, but lack of structure; confidence is uncoupled from mutation authority	B2 is closest relative of this family; B3 adds critic loops, validators and governance coupling
Retrieval-augmented generation	Retrieval-augmented and dense-retrieval methods [8–10]	Strong evidence base for grounding and factual memory recall	Varying retrieval quality; context-packing is not well defined for workflow-mutation situations	B3 converts retrieval into operation through ranking of context packets and explicit tracking of missing signals
Agent/tool orchestration	Tool-use methods and agent benchmarks [11–14]	Strong multi-step execution ability and coordination with external systems	Inconsistent run-time behavior and risk of harm [15] when there is no deterministic bound on mutation	B3 constrains execution using policy-gated, hash-checked and idempotent write-back
Trust [16], confidence controls, governance	Alignment, human-AI collaboration, automation-trust, and socio-technical SE studies [17–20]	Facilitates control and accountability [21]	Typically investigated in settings away from issue-tracker mutation	B3 directly implements this layer in Jira using human approval and policy gates

2 API and UI Console Workflow

2.1 Workflow API

The workflow has two tasks: enriching existing Jira items and breaking epics into candidate stories. Drafting, human review, and changes to Jira are separate steps. A run can cover an issue, sprint, epic, or release. Reviewers can check the run status and each item's draft, evidence, warnings, readiness, confidence, policy state, and prompt-selection details before making a decision.

For one item or the whole run, reviewers can approve, reject, request a new draft, suggest a change, or provide an instruction for further revision. Suggestions and reviewer-written instructions return to the reasoning process to produce a revised draft. If the system asks a question, the reviewer's answer continues the revision. Writing an approved item back to Jira is a separate action, allowed only after policy, validation, source-conflict, and retry checks. When several items are written together, each keeps its own result and reason for success, a block, a conflict, or a repeated request.

To break down an epic, the system reads the epic, existing child stories, and available evidence. It returns editable candidates with readiness, confidence, threshold status, warnings, and evidence references, without creating Jira stories. The creation step takes the reviewed story data, checks confidence and approval, skips matching existing summaries, and tries to create the remaining stories. It lists created and skipped items, with reasons for duplicates and creation failures. Reviewers and controlled software clients use the same API rules and governance checks throughout inspection, guidance, revision, approval, and writeback.

2.2 Console Workflow

The console has three connected views. The enrichment view starts a run and shows each draft with its evidence, warnings, readiness, confidence, and selected prompt. The review view provides the approval and revision actions, including suggestions and Jira-specific prompts, for one item or the whole run. The Epic Story Breakdown view takes epic and project keys, requests story suggestions, and shows editable, structured story data with an `approved` flag for each story. Reviewers can edit content, add missing context, or provide a new prompt before using a separate action to create approved stories that pass the required checks. A revision request does not authorize a change to Jira.

Reviewers read the stories and assess their risks through the console. Controlled agents and automated workflows use the same API with fixed rules for the submitted data. This supports integration with release-planning bots, PM dashboards, and governance data lakes under the same controls. The console shows the sources behind confidence scores, warnings, policy reason codes, approval records, duplicate-skip reasons, and story selections kept for retry. These displays are intended to help reviewers interpret confidence and choose whether to accept, reject, or retry. Their effect on live console decisions was not measured in the artifact study. The design seeks appropriate trust rather than maximum automation, reflecting evidence that human–AI team performance depends on users' understanding of system behavior [16, 18, 21].

2.3 Evidence Availability and Recovery

The system classifies warning codes, such as external authentication requirements and MCP unavailability, and propagates them to run- and story-level review records. This supports graceful degradation:

$$\texttt{service_mode} \in \{\texttt{full-context}, \texttt{reduced-context}\}. \tag{S1}$$

Full-context mode is used for cases with external evidence; *reduced-context* mode is applied when only issue-local evidence is available.

Reduced-context mode is designed to remain useful instead of failing softly. It still provides structured drafts and warning metadata, which allows reviews to continue in an informed way.

In reliability engineering, the "graceful capability collapse" pattern is applied instead of binary failure. For example, if a high-value evidence source such as Confluence is offline, the system retains decomposition capability by using the available issue records and marking the missing evidence categories. Available issue records prevent planning deadlock, while metadata preservation maintains the meaning of the confidence score.

Transient failures are retried using exponential backoff and jitter, while terminal errors are reported with machine-actionable error codes. The technique allows for automatic retry management at the orchestration layer without hiding permanent integration issues.

The user-guided workflow maintains collaboration context through the stages in case of failures. If the story creation fails or times out, then the selected stories will still be present in the console. User will be able to look at the partial generation results in Jira and continue working on them without having to regenerate the decision set. Such capability is important for human–AI cooperation since recovery should not be limited to backend functionality but should be visible for a human making retry, change, or cancel decisions.

3 System Architecture

Figure 1(a) and Figure 1(b) refer to the architecture panels in the main manuscript.

Figure 1(a) presents the complete system architecture, showing how the LLM, RL-guided prompt-selection layer, and human-governance mechanisms interact to address the software-engineering and requirements-engineering challenges discussed above. The subsequent sections explain the main architectural components in greater detail.

3.1 Reading the architecture figures.

The set of Figures 1(a) and 1(b) and Supplementary Figure S1 provides three perspectives: architecture and trust boundaries, operational sequence, and internal decomposition of agents. The deliberate overlap helps to conduct separate reviews of completeness of the architecture, correctness of the execution, and responsibilities of agents. Crucially, the interface junctions and directional arrows mapped across these graphics represent strict, programmatically enforced API contracts. System components interact across boundaries through rigid REST and schema serialization layers, ensuring that data handoffs are deterministically bound rather than relying on fluid text validation.

Figure 1(a): system architecture overview. Figure 1(a) places Jira epics, stories, and writeback targets; Confluence requirements and design evidence; and Bitbucket code and review signals outside the AI boundary. Inside Jira Enhancer, the review console, orchestrator, semantic enrichment engine, RL policy, governance layer, event store, and conflict-safe writer transform evidence into candidate stories. The architecture clearly separates AI reasoning from changes to Jira. The enrichment and RL components can generate, evaluate, and improve a candidate story, but they cannot update Jira directly. A candidate can be written to Jira only after it passes validation and the confidence threshold, complies with the relevant policies, and receives human approval.

Figure 1(b): execution/control flow. Figure 1(b) displays the components in execution order. A run gathers Jira context and acquires requirement, design, code, and review signals. The orchestrator first makes a workflow entry and selects a permitted prompt strategy. Then, it generates and scores candidate stories, applies the policies, and logs all the steps. Only successful candidates may be written to Jira. Each arrow in the diagram depicts a clearly defined handoff from one component to another, which includes preparing inputs, logging the selection of prompts, candidate scoring, logging the policies applied, and validation of writing back to Jira. Before writing to Jira, the system ensures there are no conflicting edits, safe retries (idempotency), and duplicated stories. Otherwise, the item is left in review, refinement, conflict, or failure state without Jira being updated.

Supplementary Figure S1 describes the four modules for story enhancement, epic-to-story decomposition, prompt generation, and RL-based prompt selection.

The captions in Figures 1(a) and 1(b) represent executable contracts. Context and Code Retrieve normalize the inputs, Human Actions captures the reviewer's intention, while selecting and generating safe-arms leads to scoring of candidates; Policy Check determines confidence, role, validation, and approval; Persist trace records state, results, and evidence links. Approved creation reaches controlled writeback only after the source hash, idempotency, duplicate, and target checks. The safety boundary is hence determined by this rule rather than fluency of text.

4 Traceable Data Records

Every run captures scope, requester, time stamps, and idempotency information along with item state, confidence, readiness, policy flags, structured drafts, references to external evidence, decisions and notes on approval, and before/after writeback hashes. Since each piece of information is a first class record, the planning decision behind it may be subsequently reconstructed for empirical evaluation [22, 23], auditing [24], or incident analysis. Immutable event streams maintain event ordering, whereas mutable snapshots facilitate low-latency querying. Informational warnings alert about non-critical deficiencies, advisory warnings bring problems to reviewer attention, and blocking warnings stop mutation until the issue gets fixed.

5 Epic-to-Story Decomposition Method

5.1 Candidate Generation

By leveraging epic context and previous stories, the system can produce any required capability archetypes:

- UI/interaction story,
- API/contract story,
- persistence/writeback story,
- validation/regression story.

This system rejects the archetypes that are already covered in existing summaries via lexical and semantic matching.

This helps avoid the common error of "story inflation," where assistants create tickets that differ in wording but cover the same work.

In the current implementation, the system recognizes any embedded epic description as a first-class piece of evidence. When an epic is filled with design signals such as solution-design headings, scaling, formulas, APIs and workflows, validation, roll-back considerations, and testing considerations, the system will output a `jira_embedded_context` piece of evidence. This makes the decomposition resilient to the common enterprise challenge where Confluence links are unavailable at the time of execution, while the epic still contains enough design information for bounded planning. In case when the epic has only links and those links are not available, the system avoids the generation of any placeholders and returns the context unavailability message.

5.2 Story Point Assignment

Initial estimates are derived from complexity priors:

$$\mathrm{SP}(s) = f(\xi_{\text{scope}}, \xi_{\text{integration}}, \xi_{\text{risk}}, \xi_{\text{unknown}}), \tag{S2}$$

with the four ξ factors representing normalized priors for the complexity of scope, integration, risk, and uncertainty. The resulting measure is categorized using Fibonacci-like buckets of 3, 5, and 8. In the current implementation, the default values are deterministic (3/5) and may be learned in future implementations.

Deterministic baseline is an intended choice. It guarantees consistent behavior in initial phases of use and avoids estimation variability until there is enough history to learn from. It also represents a conservative approach to evaluation whereby the value brought in by automation is compared with deterministic baseline performance before applying any other estimation techniques.

5.3 Iterative Enrichment

For each candidate story s:

1. run semantic analysis,
2. calculate readiness/confidence,
3. if $C(s) < \tau_c$, provide guidance and repeat the enrichment process within a bounded attempt budget,
4. provide a structured story payload.

Enrichment increases as each iteration brings explicit feedback from reviewers, such as missed edge cases, unresolved dependencies, or ambiguous non-functional requirements. Thus, the refinement process becomes a collaboration, not just prompting. The distinction is important for successful interaction between human and AI [18, 21].

Moreover, in one embodiment, the output of the enrichment process follows a canonical, strongly typed schema: `problem_statement`, `actor_value`, `functional_scope`, `nfr_constraints`, `acceptance_criteria`, `dependency_map`, `risk_register`, and `test_considerations`. The high degree of typing ensures interoperability of the result with tools for reviews, analytics, and policies and removes ambiguity caused by the informal nature of the free-form text.

An analysis of coverage on the epic level can be done through the capability matrix. Its rows correspond to the capability dimensions such as UI, API, data, observability, resiliency, and security, while its columns represent the suggested stories. The system is able to detect undercovered dimensions and generate candidate stories accordingly. It alters the decomposition process from merely generative to constrained generation with clear objectives.

Algorithm S1: Ablation-Mode Execution for B0/B1/B2/B3 Comparison

Input: Epic unit E_u, mode $m \in \{\text{B0}, \text{B1}, \text{B2}, \text{B3}\}$
Output: Metric row $(Q_m, \text{Eff}_m, G_m, \text{Composite}_m, \text{Time}_m)$
// Apply the selected mode's component schema.
if $m = \text{B0}$ **then**
    ExecuteManualReferenceCore(E_u)
                                                    // Independent manual package required
else
    ExecuteTemplateCore(E_u) if $m = \text{B1}$
    ExecuteGenericLLMCore(E_u) if $m = \text{B2}$
    ExecuteFullJiraEnhancerCore(E_u) if $m = \text{B3}$
// Disable B3-only components in baseline modes.
if $m \neq \text{B3}$ **then**
    Disable components excluded from mode m (critic loop / context packer / validator / governed writeback)
// Use one scorer; each row retains its measured or analytical status.
Collect metrics using the same scorer and schema across all modes
Compute Q_m (quality), Eff_m (efficiency), G_m (governance), Composite_m, and Time_m
return $(Q_m, \text{Eff}_m, G_m, \text{Composite}_m, \text{Time}_m)$

5.4 Ablation Execution Algorithm

Algorithm S1 specifies the execution-mode schema and common scorecard for the four baseline definitions. Each mode activates its listed components and returns quality, efficiency, governance, composite, and assessment-time fields.

References

[1] G. Lucassen, F. Dalpiaz, J. M. E. M. van der Werf, S. Brinkkemper, Improving agile requirements: The quality user story framework and tool, Requirements Engineering 21 (2016) 383–403.

[2] ISO/IEC/IEEE, ISO/IEC/IEEE 29148:2018 Systems and Software Engineering—Life Cycle Processes—Requirements Engineering, ISO/IEC/IEEE, 2018.

[3] L. Cao, B. Ramesh, Agile requirements engineering practices: An empirical study, IEEE Software 25 (2008) 60–67.

[4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: Advances in Neural Information Processing Systems (NeurIPS), volume 33, Curran Associates, Inc., Red Hook, NY, USA, 2020, pp. 1877–1901.

[5] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems (NeurIPS), volume 35, Curran Associates, Inc., Red Hook, NY, USA, 2022, pp. 24824–24837. doi:10.52202/068431-1800.

[6] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, J. M. Zhang, Large language models for software engineering: Survey and open problems, in: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), IEEE, Melbourne, Australia, 2023, pp. 31–53. doi:10.1109/ICSE-FoSE59343.2023.00008.

[7] S. Barke, M. B. James, N. Polikarpova, Grounded copilot: How programmers interact with code-generating models, Proceedings of the ACM on Programming Languages 7 (2023) 85–111.

[8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive nlp tasks, in: Advances in Neural Information Processing Systems (NeurIPS), volume 33, Curran Associates, Inc., Red Hook, NY, USA, 2020, pp. 9459–9474.

[9] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense passage retrieval for open-domain question answering, in: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, Online, 2020, pp. 6769–6781. URL: https://aclanthology.org/2020.emnlp-main.550/. doi:10.18653/v1/2020.emnlp-main.550.

[10] A. Asai, S. Min, Z. Zhong, D. Chen, H. Hajishirzi, W.-t. Yih, Y. Choi, M. Lewis, L. Zettlemoyer, Self-rag: Learning to retrieve, generate, and critique through self-reflection, in: International Conference on Learning Representations (ICLR), OpenReview.net, Vienna, Austria, 2024.

[11] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing Systems (NeurIPS), volume 36, Curran Associates, Inc., Red Hook, NY, USA, 2023, pp. 68539–68551. doi:10.52202/075280-2997.

[12] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, J. Tang, Agentbench: Evaluating llms as agents, in: The Twelfth International Conference on Learning Representations, OpenReview.net, Vienna, Austria, 2024. URL: https://openreview.net/forum?id=zAdUB0aCTQ.

[13] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, C. Wang, Autogen: Enabling next-gen llm applications via multi-agent conversations, in: Conference on Language Modeling (COLM), OpenReview.net, Philadelphia, PA, USA, 2024. URL: https://openreview.net/forum?id=BAakY1hNKS.

[14] J. Lu, T. Holleis, Y. Zhang, B. Aumayer, F. Nan, H. Bai, S. Ma, S. Ma, M. Li, G. Yin, Z. Wang, R. Pang, Toolsandbox: A stateful, conversational, interactive evaluation benchmark for llm tool use capabilities, in: Findings of the Association for Computational Linguistics: NAACL 2025, Association for Computational Linguistics, Albuquerque, New Mexico, 2025, pp. 1160–1183. doi:10.18653/v1/2025.findings-naacl.65.

[15] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, T. Hashimoto, Identifying the risks of lm agents with an lm-emulated sandbox, in: The Twelfth International Conference on Learning Representations, OpenReview.net, Vienna, Austria, 2024. URL: https://openreview.net/forum?id=GEcwtMk1uA.

[16] J. D. Lee, K. A. See, Trust in automation: Designing for appropriate reliance, Human Factors 46 (2004) 50–80.

[17] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, R. Lowe, Training language models to follow instructions with human feedback, in: Advances in Neural Information Processing Systems (NeurIPS), volume 35, Curran Associates, Inc., Red Hook, NY, USA, 2022, pp. 27730–27744. doi:10.52202/068431-2011.

[18] G. Bansal, B. Nushi, E. Kamar, W. S. Lasecki, D. S. Weld, E. Horvitz, Beyond accuracy: The role of mental models in human-ai team performance, in: Proceedings of the AAAI Conference on Human Computation and Crowdsourcing, AAAI Press, Palo Alto, CA, USA, 2019, pp. 2–11.

[19] R. Parasuraman, T. B. Sheridan, C. D. Wickens, A model for types and levels of human interaction with automation, IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans 30 (2000) 286–297.

[20] M.-A. Storey, N. A. Ernst, C. Williams, E. Kalliamvakou, The who, what, how of software engineering research: A socio-technical framework, Empirical Software Engineering 25 (2020) 4097–4129.

[21] S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, E. Horvitz, Guidelines for human-ai interaction, in: Proceedings of the CHI Conference on Human Factors in Computing Systems, Association for Computing Machinery, New York, NY, USA, 2019, pp. 1–13.

[22] B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, J. Rosenberg, Preliminary guidelines for empirical research in software engineering, IEEE Transactions on Software Engineering 28 (2002) 721–734.

[23] P. Runeson, M. Höst, Guidelines for conducting and reporting case study research in software engineering, Empirical Software Engineering 14 (2009) 131–164.

[24] L. Moreau, P. Missier, PROV-DM: The PROV Data Model, W3C Recommendation, World Wide Web Consortium, 2013. URL: https://www.w3.org/TR/prov-dm/.